

Automated CI/CD Deployment of a Containerised App on AWS

The goal of this project is to automate the deployment of a containerised Node.js web application.

The application is first packaged into a Docker image, stored in Amazon Elastic Container Registry (ECR), and then deployed to a Kubernetes cluster. Jenkins is used to automate the complete workflow, from pulling the latest code from GitHub to deploying the updated application on Kubernetes.

Project Architecture

The project consists of the following components:

- Linux Host (Killercode)
- AWS IAM User
- AWS Elastic Container Registry (ECR)
- Node.js Express Application
- Docker
- Kubernetes Deployment & Service
- GitHub Repository
- Jenkins Pipeline

Prerequisites

Before starting, make sure you have:

- A Killercode Kubernetes playground (or any Linux machine with Kubernetes configured)
- An AWS account
- A GitHub account
- Basic knowledge of Linux commands

Step 1: Create AWS Resources

Where: AWS Console

The first step is to create the AWS resources required by Jenkins and Docker.

1.1 Create an IAM User

Navigate to:

IAM → Users → Create User

Create a user named:

Create an Access Key for programmatic access.

Attach the following policy:

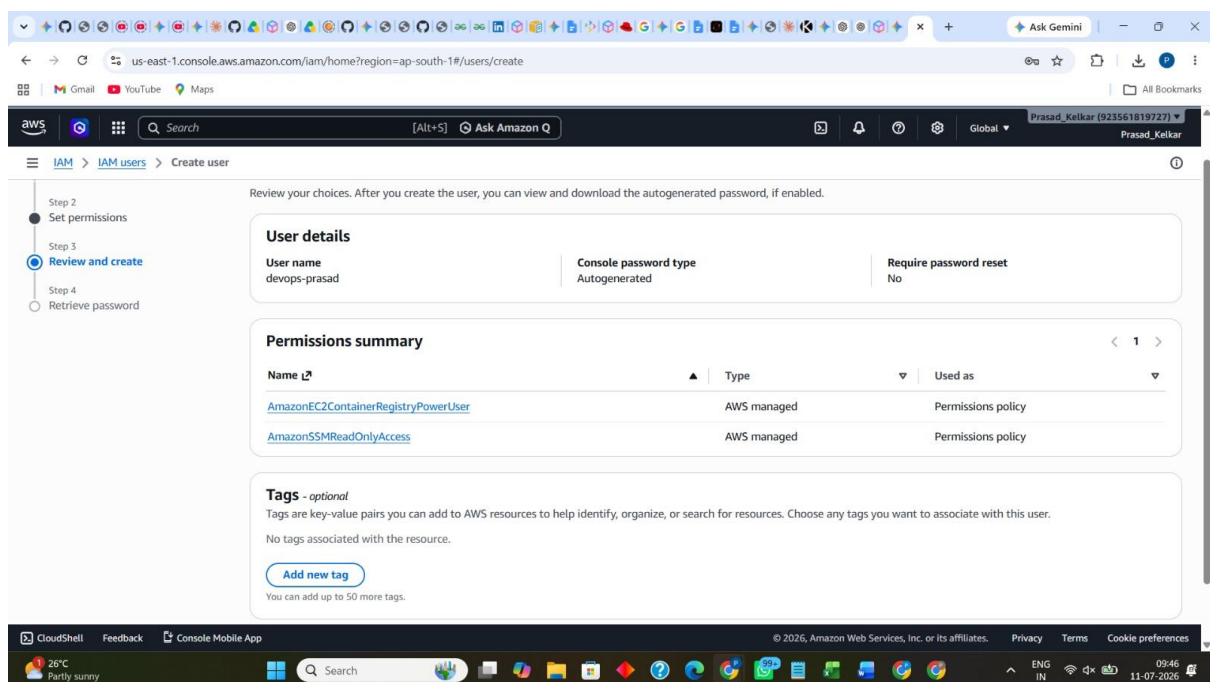
AmazonEC2ContainerRegistryFullAccess

Save the following details because AWS shows them only once:

- Access Key ID
- Secret Access Key

Purpose

This IAM user allows Jenkins and Docker to securely authenticate with AWS and push Docker images to Amazon ECR.



1.2 Create an ECR Repository

Navigate to:

Amazon ECR → Repositories → Create Repository

Repository Name:

prasad-web-app

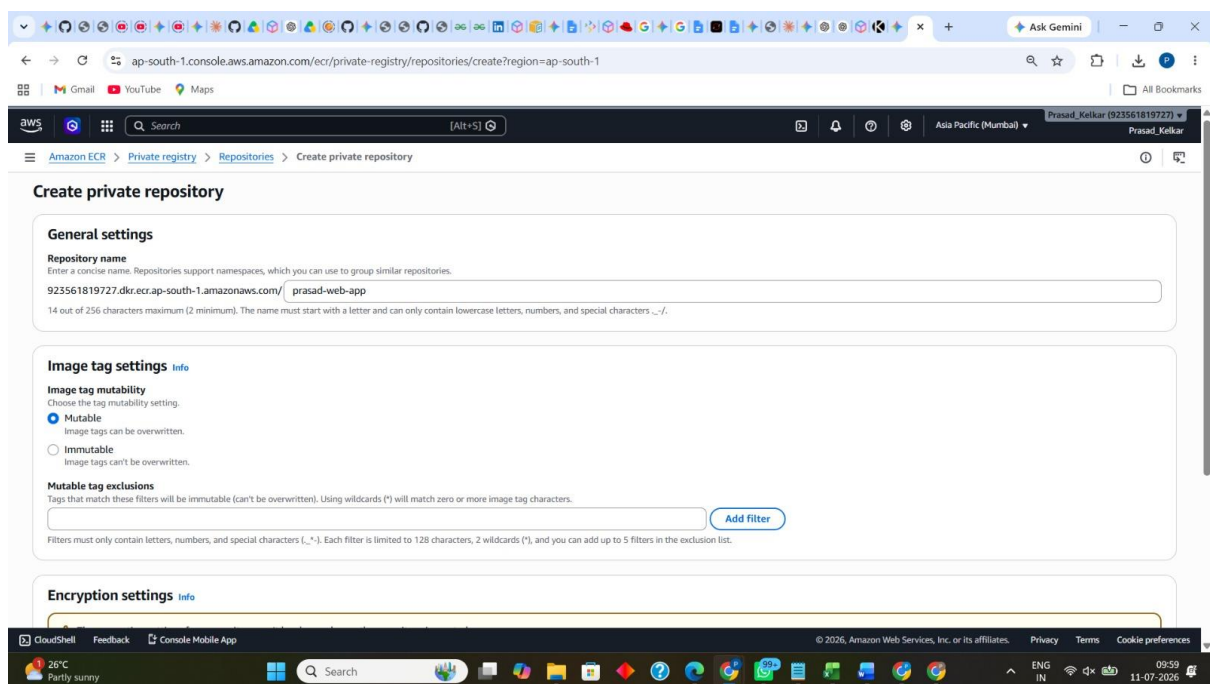
Region:

ap-south-1 (Mumbai)

Keep the repository private.

Purpose

Amazon ECR stores Docker images that Kubernetes later downloads and runs.



Step 2: Prepare the Linux Host

Create a dedicated user and perform some basic system hardening.

sudo useradd -m -s /bin/bash devops

**echo "devops ALL=(ALL) NOPASSWD:ALL" | sudo tee
/etc/sudoers.d/devops**

```
sudo firewall-cmd --permanent --add-port={8080/tcp,22/tcp,30080/tcp}
```

```
sudo firewall-cmd --reload
```

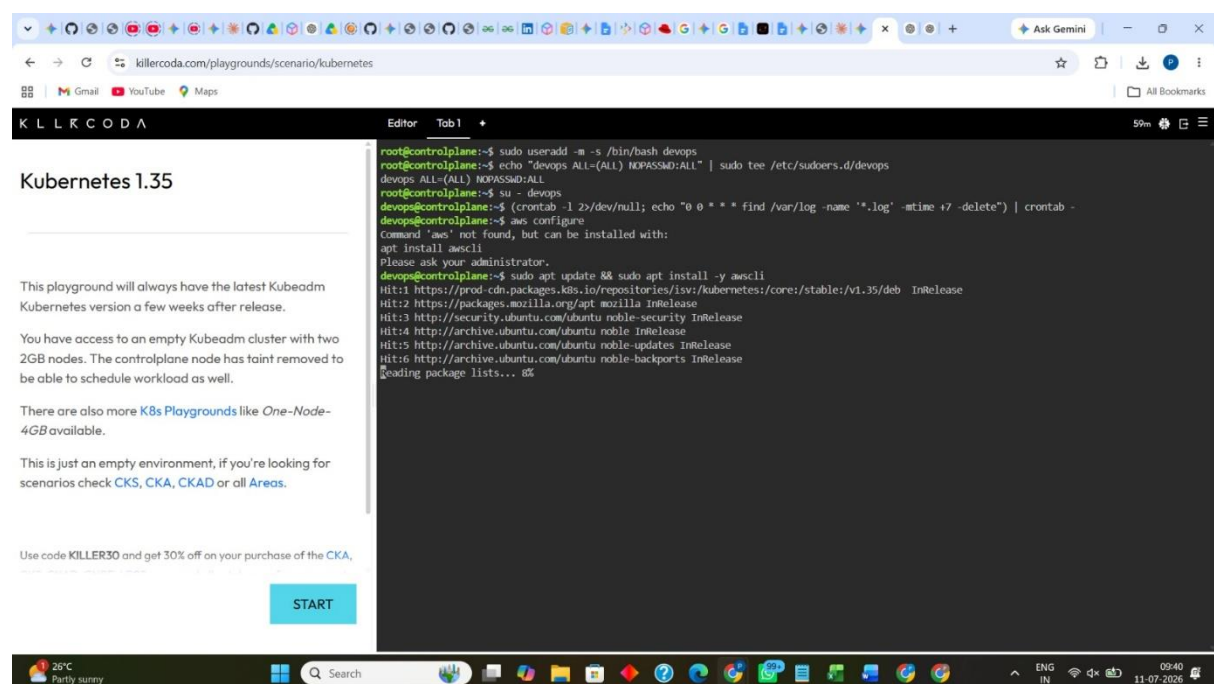
```
(crontab -l 2>/dev/null; echo "0 0 * * * find /var/jenkins_home/logs/ -name '*.log' -mtime +7 -delete") | crontab -
```

su - devops

Purpose

This step:

- Creates a separate user for DevOps work.
- Allows passwordless sudo.
- Opens only the required ports.
- Automatically removes old Jenkins log files.



Step 3: Configure AWS CLI

Run:

```
aws configure
```

Enter:

AWS Access Key ID

AWS Secret Access Key

Region : ap-south-1

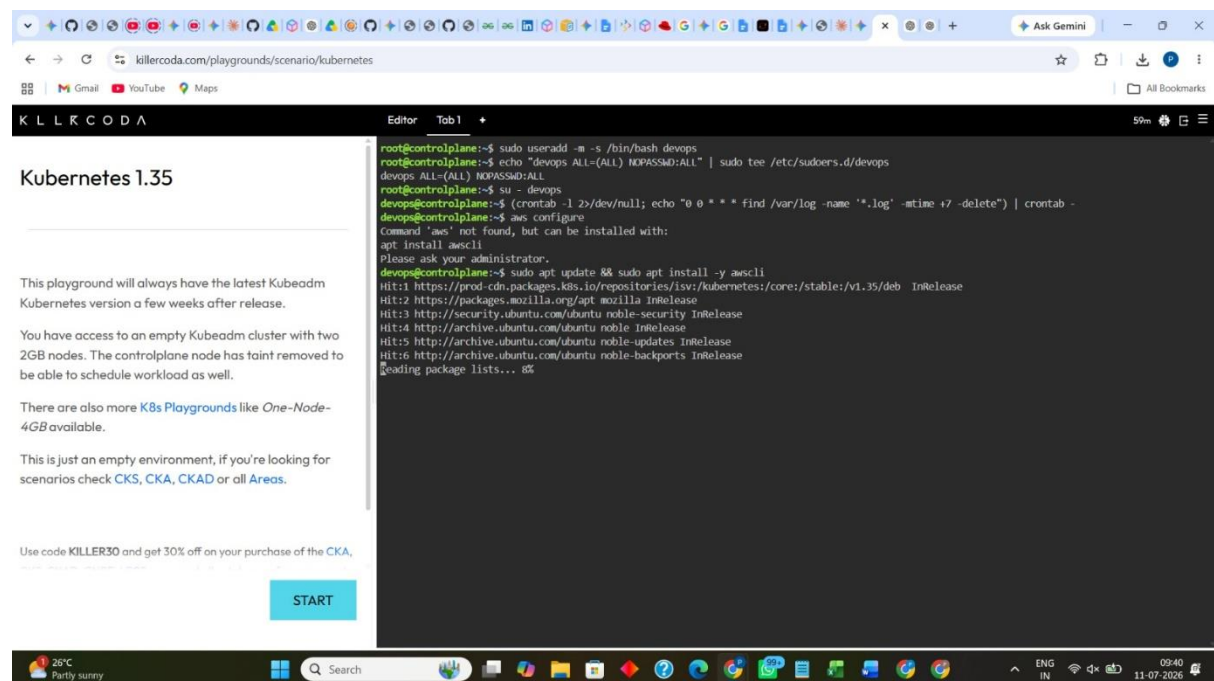
Output : json

Verify the configuration:

aws sts get-caller-identity

Purpose

This stores AWS credentials locally so Docker and Jenkins can communicate with AWS.



Step 4: Create the Application Files

Create a project directory.

mkdir -p ~/workspace

cd ~/workspace

Create the following files:

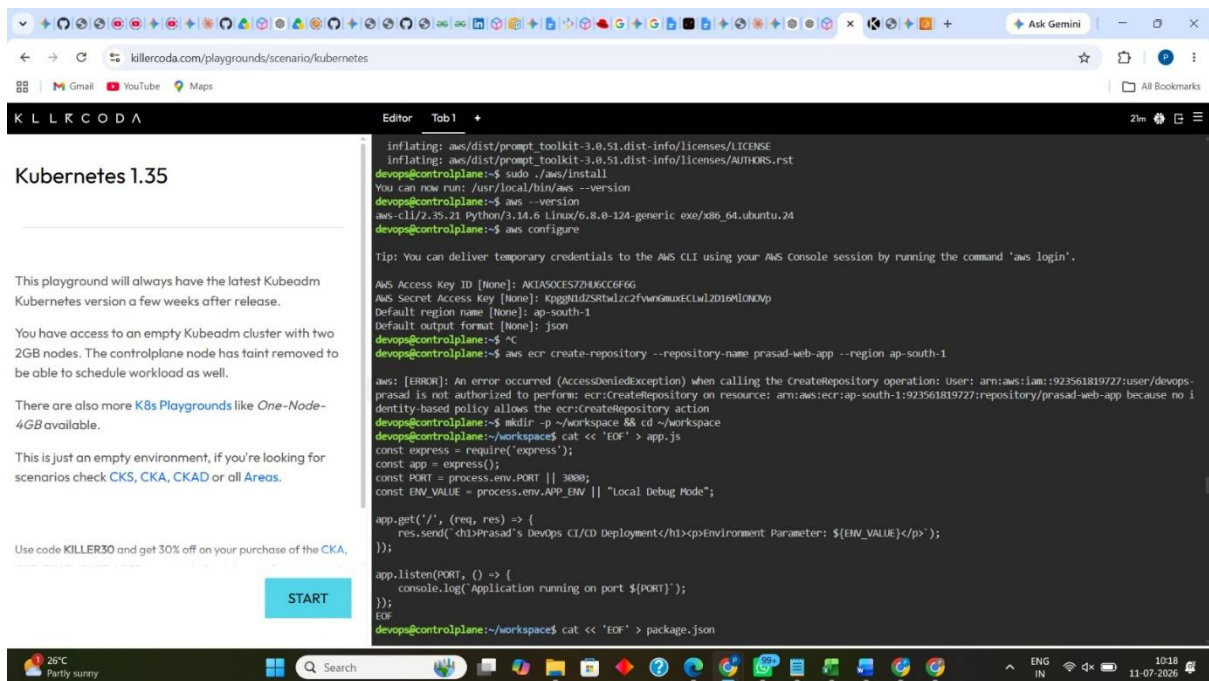
- Dockerfile

- app.js
- package.json
- deployment.yaml

Purpose

These files define:

- the Node.js application,
- the Docker image,
- and the Kubernetes deployment.



Step 5: Build and Push Docker Image

Build the Docker image.

sudo docker build -t prasad-web-app:1.0.0 .

Tag the image.

**sudo docker tag prasad-web-app:1.0.0 **

923561819727.dkr.ecr.ap-south-1.amazonaws.com/prasad-web-app:1.0.0

Login to Amazon ECR.

```
aws ecr get-login-password --region ap-south-1 | sudo docker login --  
username AWS --password-stdin 923561819727.dkr.ecr.ap-south-  
1.amazonaws.com
```

Push the image.

```
sudo docker push 923561819727.dkr.ecr.ap-south-  
1.amazonaws.com/prasad-web-app:1.0.0
```

Purpose

This confirms that the Docker image builds successfully and can be uploaded to Amazon ECR.

Step 6: Deploy the Application on Kubernetes

Create the Kubernetes image pull secret.

```
kubectl delete secret aws-ecr-secret --ignore-not-found
```

```
kubectl create secret docker-registry aws-ecr-secret \  
--docker-server=923561819727.dkr.ecr.ap-south-1.amazonaws.com \  
--docker-username=AWS \  
--docker-password=$(aws ecr get-login-password --region ap-south-1) \  
--docker-email=prasad@example.com
```

Deploy the application.

```
kubectl apply -f deployment.yaml
```

Verify.

```
kubectl get pods,svc -o wide
```

Purpose

The secret allows Kubernetes to download images from the private ECR repository.

Step 7: Push the Project to GitHub

Initialize Git.

```
git init
```

```
git add .
```

```
git commit -m "Project 1 initial commit"
```

Create a repository on GitHub.

Connect it.

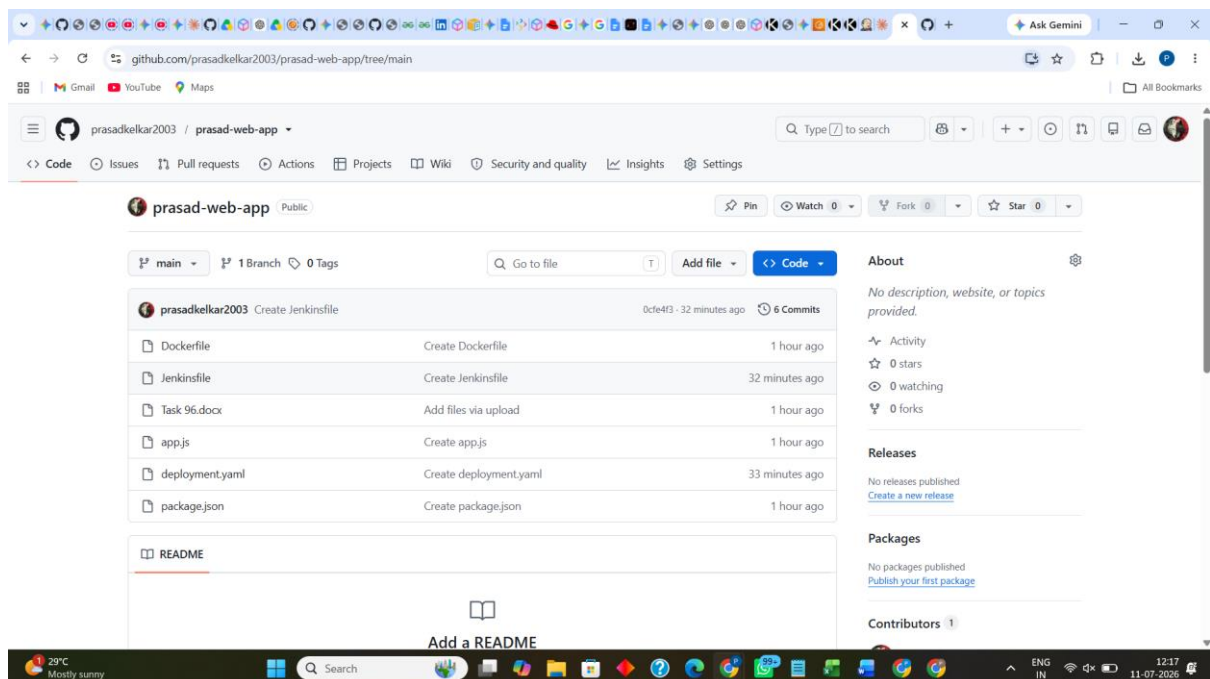
```
git branch -M main
```

```
git remote add origin https://github.com/<username>/prasad-web-app.git
```

```
git push -u origin main
```

Purpose

GitHub stores the project source code. Jenkins will download the latest code from this repository whenever the pipeline runs.



Step 8: Install Jenkins

Where: Killercoda Terminal

Run:

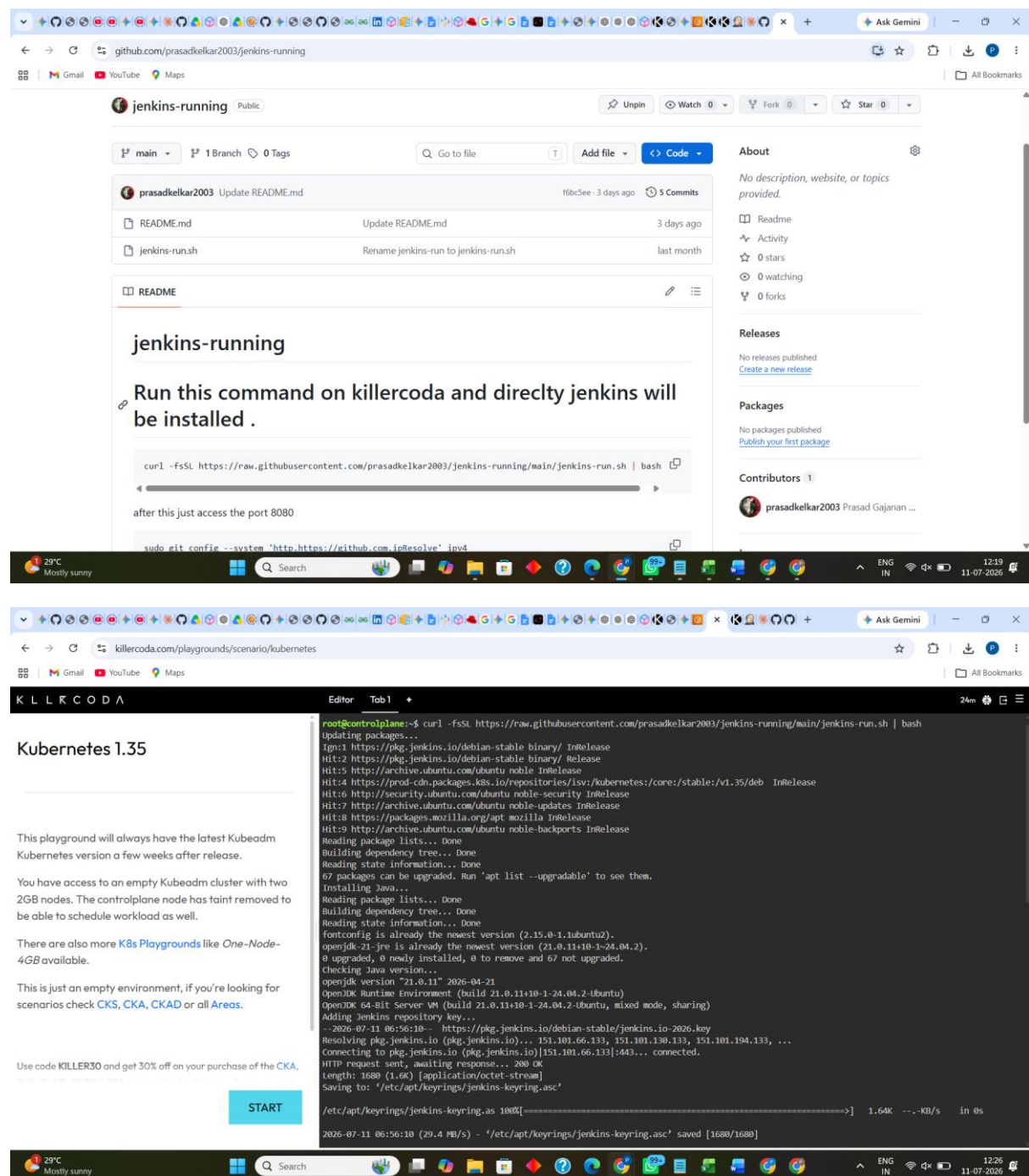
```
curl -fsSL https://raw.githubusercontent.com/prasadkelkar2003/jenkins-running/main/jenkins-run.sh | bash
```


Purpose

This installs:

- Java
- Jenkins
- Jenkins Service

The script also starts Jenkins automatically.



Step 9: Allow Jenkins to Use Docker

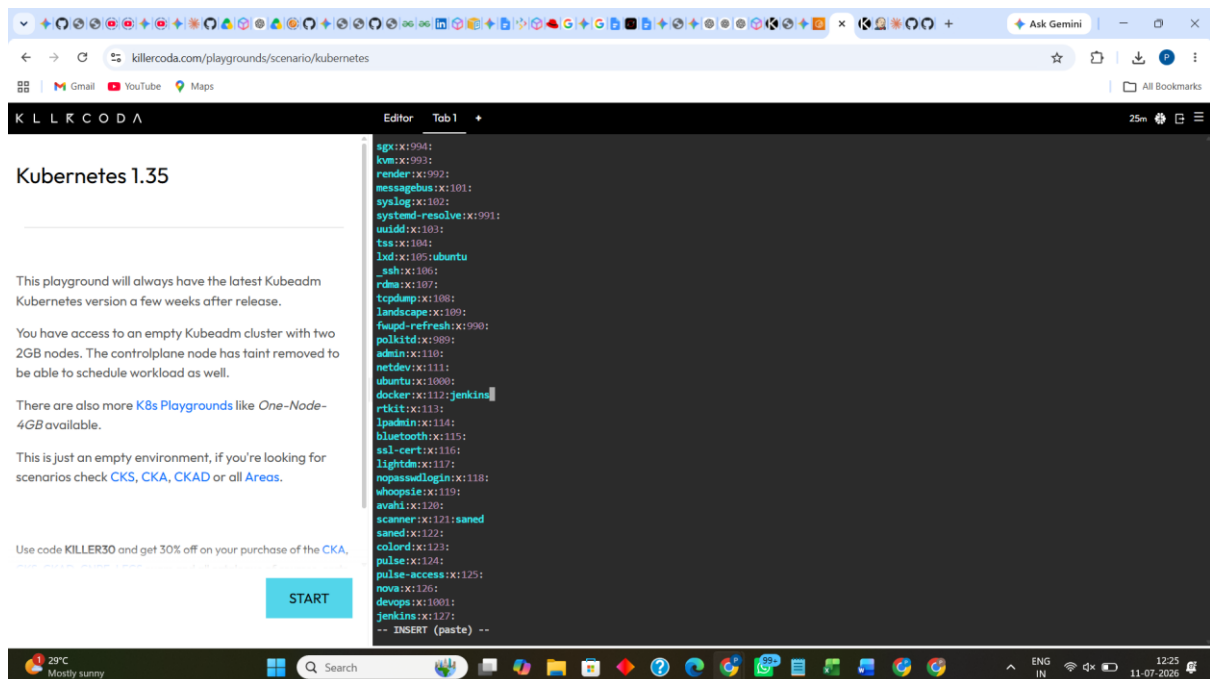
sudo usermod -aG docker jenkins

sudo systemctl restart jenkins

Purpose

Jenkins needs permission to build Docker images.

Without this step, Docker commands inside Jenkins will fail.



Step 10: Complete Jenkins Setup

Where: Jenkins Web Interface

Open Jenkins using port **8080**.

Retrieve the administrator password.

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Complete the setup wizard.

- Install Suggested Plugins
- Create an Admin User

Purpose

This completes the Jenkins installation and prepares it for pipeline creation.

Step 11: Create the Jenkins Pipeline

Inside the project directory, create a file named:

Jenkinsfile

Paste the pipeline code into the file.

Commit the changes.

git add Jenkinsfile

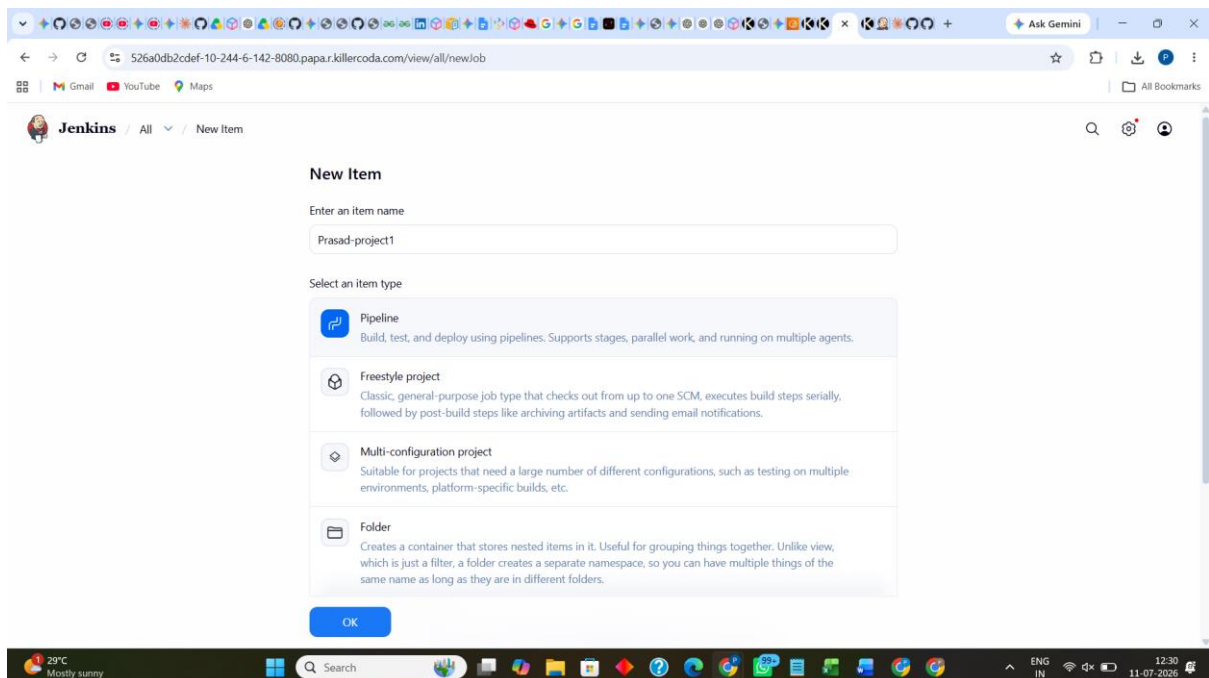
```
git commit -m "Add Jenkinsfile"
```

```
git push
```

Purpose

The Jenkinsfile defines the complete CI/CD pipeline, including:

- Source Code Checkout
- Docker Build
- Docker Push
- Kubernetes Deployment



Step 12: Create a Jenkins Job

Where: Jenkins Dashboard

Create a new Pipeline project.

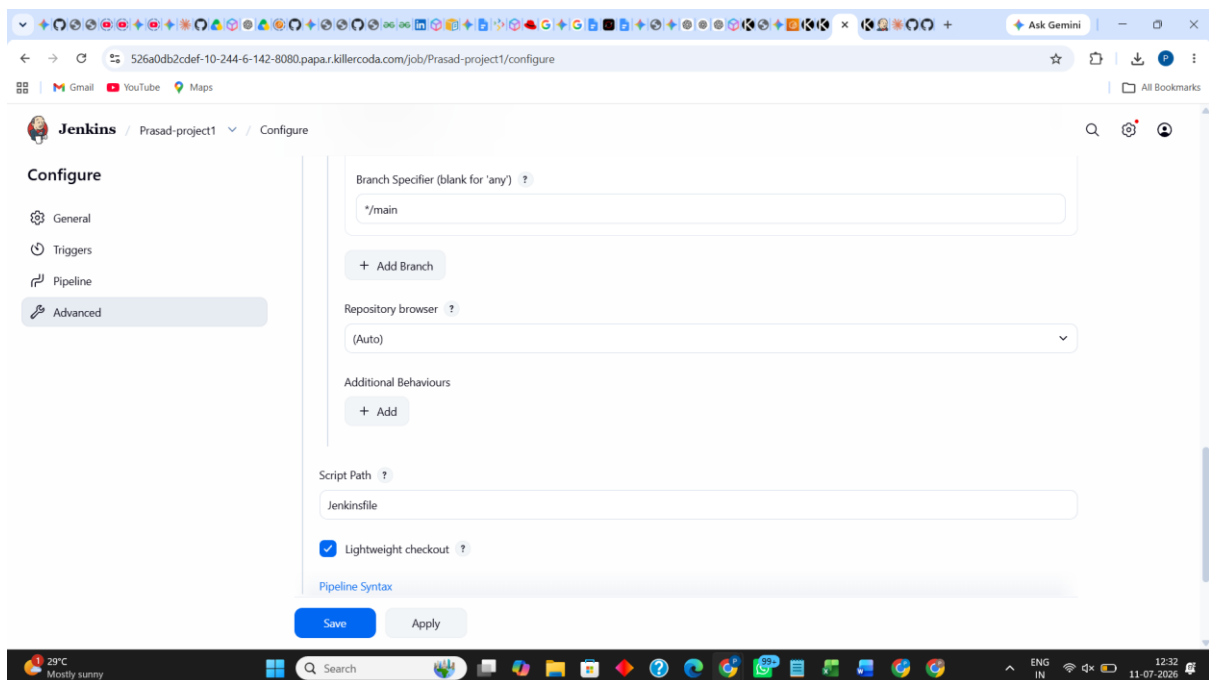
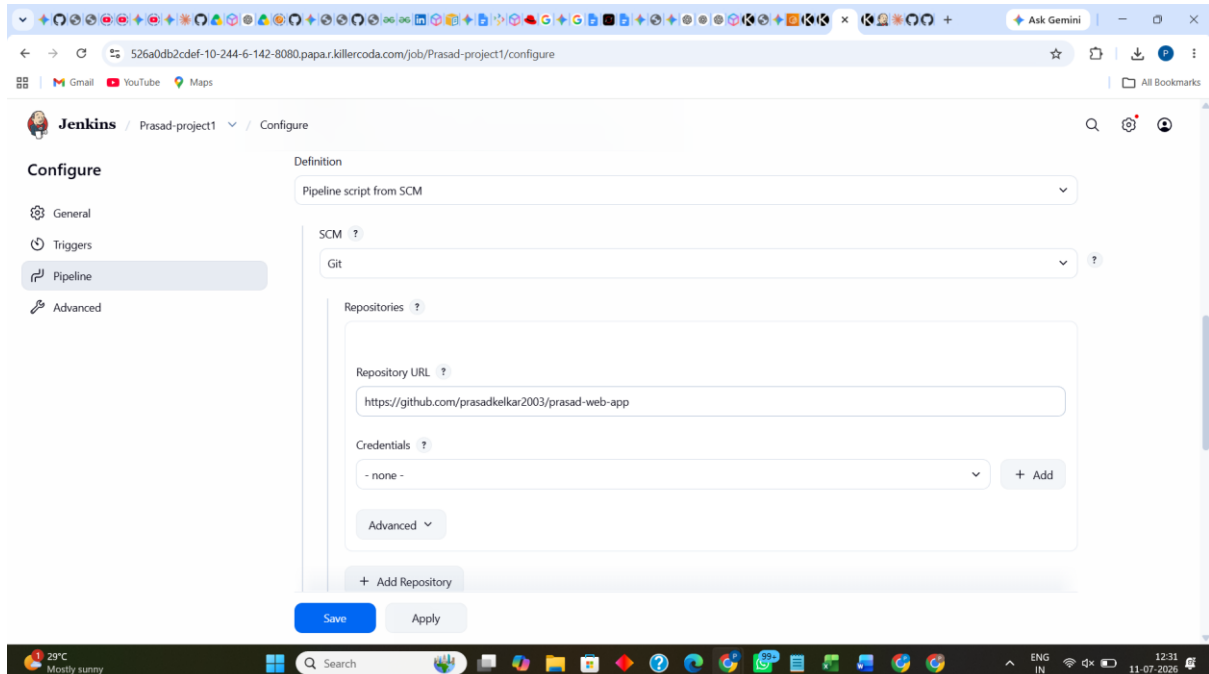
Configure:

- Pipeline from SCM
- Git Repository URL
- Branch: main
- Script Path: Jenkinsfile

Save the job.

Purpose

This tells Jenkins where the project is stored and how to execute the pipeline.



Step 13: Run the Pipeline

Open the Jenkins project.

Click:

Build Now

Monitor the Console Output.

The pipeline performs:

- Checkout Source Code
- Build Docker Image
- Push Image to Amazon ECR
- Deploy to Kubernetes

Purpose

Instead of manually running multiple commands, Jenkins completes the entire deployment automatically.

Step 14: Verify the Deployment

Check the Kubernetes resources.

kubectrl get pods,svc -o wide

Test the application.

curl http://localhost:30080

You can also open port **30080** from the Killercode Traffic panel.

Expected Output

The browser should display:

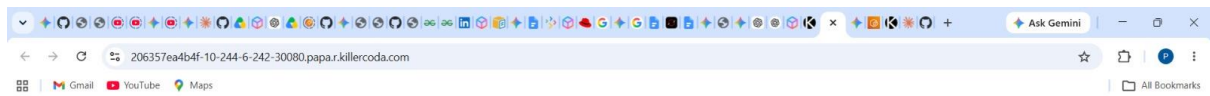
Prasad's DevOps CI/CD Deployment

Environment Parameter:

AWS-Killercode-Pipeline-Active

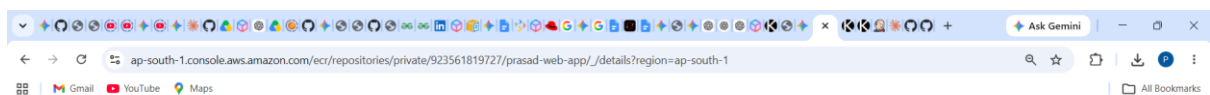
Purpose

This confirms that the application was successfully built, pushed to ECR, deployed on Kubernetes, and is accessible through the NodePort service.



Prasad's DevOps CI/CD Deployment

Environment Parameter: AWS-Killercode-Pipeline-Active



Amazon Elastic Container Registry

Private registry

Repositories

prasad-web-app

Features & Settings

Public registry

Repositories

Settings

ECR public gallery

Amazon ECS

Amazon EKS

Getting started

Documentation

prasad-web-app

Summary Images Lifecycle policy Permissions Repository tags

Images (1) Info

Filter active images

Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
1.0.0	Image	July 11, 2026, 10:05:06 (UTC+05.5)	45.61	sha256:e05bb4557c...	July 11, 2026, 10:12:30 (UTC+05.5)

Buttons: Delete, Copy URI, Details, Scan, View push commands

